

Project: Esternalità di rete e dinamiche di adozione

diffusione di tecnologie, piattaforme digitali e metodi computazionali

1. Obiettivi della dispensa

Questa dispensa introduce un modello semplice ma molto utile per studiare l'adozione di beni e servizi soggetti a esternalità di rete. Il caso tipico è quello di piattaforme digitali, sistemi di comunicazione, standard tecnologici, software, social media e servizi il cui valore per il singolo utente cresce al crescere del numero di altri utenti già presenti.

Gli obiettivi sono cinque:

1. formalizzare il legame tra utilità individuale e diffusione aggregata;
2. derivare la condizione di equilibrio di mercato in presenza di eterogeneità tra gli utenti;
3. discutere la possibilità di equilibri multipli e fenomeni di lock-in;
4. mostrare come il modello possa essere studiato con metodi numerici molto semplici;
5. usare il modello come base per un laboratorio computazionale.

Dal punto di vista didattico, questo caso di studio è particolarmente efficace perché collega economia, diffusione sociale, sistemi complessi e simulazione numerica.

2. Motivazione economica e interdisciplinare

In molti mercati il valore di un bene non dipende solo dalle sue caratteristiche intrinseche, ma anche da quanti altri individui lo utilizzano. Se una tecnologia è compatibile con quella degli altri, se permette comunicazione, se genera standard comuni o se produce accesso a una rete di utenti, allora l'utilità privata cresce con la diffusione complessiva.

Esempi classici:

- telefono;
- fax;

- sistemi operativi;
- piattaforme di pagamento;
- social network;
- marketplace digitali;
- strumenti di messaggistica;
- standard tecnologici.

In tutti questi casi, la decisione individuale di adozione dipende da una variabile aggregata, cioè dalla quota di mercato già penetrata dal prodotto. Di conseguenza, la domanda non è più semplicemente una funzione decrescente del prezzo. Può diventare non monotona e presentare soglie, trappole di coordinamento e molteplicità di equilibri.

3. Variabile aggregata di adozione

Supponiamo che il mercato contenga N potenziali utenti. Se M individui hanno adottato il bene o il servizio, definiamo la penetrazione di mercato come

$$q = \frac{M}{N},$$

dove $q \in [0, 1]$ rappresenta la quota di utenti che hanno adottato il prodotto.

Il caso $q = 0$ corrisponde ad assenza totale di diffusione, mentre $q = 1$ corrisponde a piena penetrazione del mercato.

Questa variabile aggregata sarà il cuore del modello, perché la convenienza ad adottare dipenderà da q .

4. Il modello base

4.1 Utilità individuale

Supponiamo che l'utilità monetaria percepita da un utente abbia la forma

$$\mu(q) = \beta f(q),$$

dove:

- β è una variabile individuale che misura la predisposizione o la disponibilità a valorizzare il servizio;
- $f(q)$ è una funzione crescente della diffusione aggregata;
- la crescita di $f(q)$ cattura l'esternalità di rete.

Assumiamo quindi

$$f'(q) > 0.$$

La funzione $f(q)$ e' uguale per tutti gli individui, mentre l'eterogeneità degli utenti e' assorbita in β .

L'interpretazione economica e' semplice: tutti beneficiano del fatto che la rete cresca, ma non tutti nello stesso modo. Alcuni utenti attribuiscono piu' valore del servizio alla crescita della rete, altri meno.

4.2 Regola di adozione

Dato un prezzo p , un individuo adotta se la sua utilità e' almeno pari al prezzo:

$$\mu(q) \geq p.$$

Poiche' $\mu(q) = \beta f(q)$, la condizione di adozione diventa

$$\beta f(q) \geq p.$$

Equivalentemente,

$$\beta \geq \frac{p}{f(q)}.$$

Questa e' una regola di soglia: a prezzo dato, adottano tutti gli individui con parametro β sufficientemente alto.

4.3 Equilibrio aggregato

Se $\rho(\beta)$ e' la densità della popolazione, la quota di utenti che adottano deve coincidere con la probabilità che la condizione precedente sia soddisfatta. Dunque, in equilibrio,

$$q = \Pr[\beta f(q) \geq p].$$

Questa e' la condizione centrale del modello. Riscrivendola in forma integrale si ottiene

$$q = \int_{\beta: \beta f(q) \geq p} \rho(\beta) d\beta.$$

Poiche' la soglia e'

$$\beta \geq \frac{p}{f(q)},$$

si può scrivere

$$q = \int_{p/f(q)}^{\infty} \rho(\beta) d\beta.$$

Questa è una equazione auto-consistente o di punto fisso: la quota di adozione q compare sia a sinistra sia a destra.

5. Caso esplicito: eterogeneità uniforme

Per ottenere formule chiuse, assumiamo che

$$\beta \sim U[0, B].$$

Allora la densità è

$$\rho(\beta) = \frac{1}{B}, \quad 0 \leq \beta \leq B.$$

La quota di individui che adottano è

$$q = \Pr \left[\beta \geq \frac{p}{f(q)} \right].$$

Se la soglia cade nell'intervallo $[0, B]$, otteniamo

$$q = \int_{p/f(q)}^B \frac{d\beta}{B}.$$

Quindi

$$q = \frac{B - p/f(q)}{B} = 1 - \frac{p}{Bf(q)}.$$

Equivalentemente,

$$p = B(1 - q)f(q).$$

Questa è la relazione implicita tra prezzo e penetrazione del mercato.

6. Esempio lineare

Supponiamo ora che

$$f(q) = q.$$

Allora la relazione di domanda implicita diventa

$$p = Bq(1 - q).$$

Questa e' una parabola, non una curva di domanda monotona decrescente nel senso classico.

Questo fatto e' molto importante. Per valori bassi di q , un aumento della diffusione rende il servizio piu' desiderabile e puo' aumentare la disponibilita' a pagare. Solo dopo un certo punto prevale l'effetto della saturazione del mercato.

Quindi, in presenza di esternalita' di rete, la domanda aggregata puo' avere una forma non monotona.

7. Interpretazione economica

Il modello dice una cosa molto semplice ma molto potente: l'adozione e' un problema di coordinamento.

Se pochi hanno adottato, il valore del bene puo' essere troppo basso per convincere la maggior parte degli utenti. Se invece la rete supera una certa soglia critica, il valore percepito aumenta e il processo di adozione puo' auto-rinforzarsi.

Questo meccanismo e' alla base di molti fenomeni osservabili:

- decollo lento seguito da rapida diffusione;
- fallimento di tecnologie valide ma prive di massa critica;
- lock-in su standard dominanti;
- dipendenza dalle condizioni iniziali;
- vantaggi strategici del first mover;
- importanza di sussidi iniziali, versioni gratuite o politiche promozionali.

8. Equilibrio come problema di punto fisso

Dal punto di vista dei metodi computazionali, il cuore del problema e' risolvere l'equazione

$$q = T(q),$$

dove

$$T(q) = \Pr[\beta f(q) \geq p].$$

Nel caso uniforme,

$$T(q) = 1 - \frac{p}{Bf(q)},$$

quando il termine ha senso ed e' compreso in $[0, 1]$.

Quindi gli equilibri di mercato sono i punti fissi della mappa T .

Questo rende il modello perfetto per introdurre:

- algoritmi di iterazione di punto fisso;
- rappresentazione grafica di $T(q)$ e della retta q ;
- studio della stabilità locale;
- dipendenza dal valore iniziale q_0 .

9. Equilibri multipli

In presenza di forti esternalità di rete, il modello puo' ammettere piu' soluzioni della condizione di equilibrio.

Dal punto di vista grafico, questo accade quando la curva

$$q \mapsto T(q)$$

interseca la retta identità in piu' punti.

In termini economici, significa che a uno stesso prezzo possono corrispondere:

- un equilibrio a bassa adozione;
- un equilibrio intermedio, spesso instabile;
- un equilibrio ad alta adozione.

Questa molteplicità di equilibri e' cruciale per comprendere fenomeni di coordinamento, tipping e irreversibilità.

10. Comparativa statica

Il modello consente una comparativa statica molto naturale.

10.1 Effetto del prezzo

A parità di tutto il resto, un aumento di p rende piu' difficile soddisfare la soglia di adozione e tende a ridurre q .

10.2 Effetto della distribuzione di β

Se la popolazione contiene piu' individui con valori alti di β , la quota di adozione tende ad aumentare.

10.3 Effetto della forza dell'esternalità

Se $f(q)$ cresce piu' rapidamente, allora l'interdipendenza strategica tra utenti e' piu' forte. Questo puo' accentuare la non linearità della dinamica e la possibilità di equilibri multipli.

10.4 Effetto delle condizioni iniziali

In una dinamica iterativa, il valore iniziale q_0 puo' determinare a quale equilibrio il sistema converge. Questo e' un elemento centrale per applicazioni a piattaforme digitali e processi di diffusione sociale.

11. Una dinamica di adozione molto semplice

Il modello statico puo' essere trasformato in una dinamica discreta:

$$q_{t+1} = T(q_t).$$

Questa equazione descrive una dinamica di aggiustamento in cui, a ogni passo, la quota di adozione si aggiorna in base alla frazione di utenti che trova conveniente adottare dato il livello corrente di penetrazione.

Nel caso uniforme, la dinamica diventa

$$q_{t+1} = 1 - \frac{p}{Bf(q_t)}.$$

Questa dinamica e' particolarmente utile in laboratorio, perche' permette di confrontare:

- convergenza rapida;
- convergenza lenta;
- soglie critiche;
- sensibilità alle condizioni iniziali.

12. Versione agent-based

Lo stesso modello puo' essere letto in chiave microscopica.

Supponiamo di avere N agenti. Ogni agente i riceve un parametro individuale β_i estratto da una distribuzione data. A ogni passo temporale, l'agente adotta se

$$\beta_i f(q_t) \geq p,$$

dove q_t e' la quota di agenti adottanti al tempo t .

Allora il sistema evolve secondo:

1. si osserva la quota corrente q_t ;
2. ogni agente valuta la soglia;
3. si aggiorna lo stato di adozione;
4. si calcola la nuova quota q_{t+1} .

Questa formulazione e' ideale per una simulazione Monte Carlo e permette di confrontare il limite di grande popolazione con sistemi finiti.

13. Pseudocodice del modello aggregato

Di seguito una versione molto semplice del problema di punto fisso.

Input

- prezzo p
- parametro massimo B
- funzione di rete $f(q)$
- valore iniziale q_0
- tolleranza numerica ε
- numero massimo di iterazioni T

Pseudocodice

1. inizializza $q = q_0$
2. per $t = 1, \dots, T$:

- calcola

$$q_{new} = 1 - \frac{p}{Bf(q)}$$

nel caso uniforme

- tronca eventualmente il valore nell'intervallo $[0, 1]$
- se

$$|q_{new} - q| < \varepsilon$$

arresta l'algoritmo

- altrimenti poni

$$q = q_{new}$$

3. restituisci q

Questo e' un algoritmo elementare di iterazione di punto fisso.

14. Pseudocodice della versione agent-based

1. genera N valori individuali β_i
2. inizializza uno stato iniziale di adozione e calcola q_0
3. per ogni passo temporale:
 - per ogni agente i :
 - se

$$\beta_i f(q_t) \geq p$$

allora l'agente adotta

- aggiorna la quota

$$q_{t+1} = \frac{\text{numero di adottanti}}{N}$$

4. salva la traiettoria temporale di q_t

Questa versione e' molto istruttiva, perche' rende visibile il collegamento tra regola individuale e risultato aggregato.

15. Schema del laboratorio

15.1 Laboratorio 1 - Curva di domanda implicita

Obiettivo

Studiare la relazione tra prezzo e penetrazione nel caso uniforme.

Attivita'

1. fissare B
2. scegliere una funzione $f(q)$
3. risolvere numericamente la condizione di equilibrio per diversi valori di p
4. rappresentare graficamente la relazione p contro q

Domande guida

- la curva di domanda e' monotona?
- in quali regioni il prezzo e la penetrazione crescono insieme?
- come cambia il risultato se si sostituisce $f(q) = q$ con una funzione diversa?

15.2 Laboratorio 2 - Equilibri multipli

Obiettivo

Individuare regioni dei parametri in cui esistono piu' punti fissi.

Attivita'

1. fissare una famiglia di funzioni $f(q)$
2. variare il prezzo
3. rappresentare $T(q)$ e la retta q
4. contare il numero di intersezioni

Domande guida

- esistono soglie critiche di prezzo?
- quale equilibrio e' stabile?
- quanto conta il valore iniziale q_0 ?

15.3 Laboratorio 3 - Simulazione agent-based

Obiettivo

Confrontare il modello aggregato con una simulazione a popolazione finita.

Attivita'

1. generare N agenti con eterogeneità β_i
2. simulare l'aggiornamento nel tempo
3. confrontare la traiettoria media con quella prevista dalla mappa aggregata
4. ripetere l'esperimento per diversi valori di N

Domande guida

- quanto contano gli effetti di popolazione finita?
- la dinamica media replica il punto fisso teorico?
- quali differenze emergono quando N e' piccolo?

15.4 Laboratorio 4 - Politiche di attivazione della rete

Obiettivo

Studiare come una politica iniziale possa spingere il sistema oltre una soglia critica.

Attivita'

1. fissare un prezzo e un modello con possibile equilibrio multiplo
2. introdurre un seme iniziale di adottanti
3. confrontare la dinamica con e senza seme iniziale
4. interpretare il risultato come politica di sussidio o freemium

Domande guida

- esiste una massa critica minima?
- quanto deve essere grande il seme iniziale?
- una riduzione temporanea del prezzo puo' spostare il sistema verso l'equilibrio alto?

16. Estensioni naturali

Il modello base e' volutamente semplice. Le sue estensioni piu' naturali sono molte.

1. **Funzioni di rete non lineari:** Invece di $f(q) = q$, si possono studiare funzioni concave, convesse o sigmoidi.
2. **Prezzi dinamici:** Il prezzo puo' diventare una variabile nel tempo, introducendo strategie di penetrazione del mercato.
3. **Reti esplicite:** Invece di dipendere solo dalla quota aggregata q , l'utilità di ciascun agente puo' dipendere dalla quota di vicini che hanno adottato. In questo caso il modello si avvicina a una dinamica di contagio su rete.
4. **Adozione reversibile:** Si puo' consentire agli utenti di abbandonare il servizio se la rete si riduce o se il valore percepito scende sotto una soglia.
5. **Competizione tra piattaforme:** Si possono introdurre due tecnologie o due piattaforme concorrenti con esternalità di rete incompatibili. Questa e' una estensione molto importante per applicazioni a standard tecnologici e mercati digitali.

17. Perche' questo modello e' un buon caso di studio

Dal punto di vista del corso, questo modello ha molti vantaggi.

Primo, e' matematicamente semplice ma concettualmente ricco.

Secondo, permette di usare strumenti computazionali molto accessibili:

- iterazione di punto fisso;
- analisi grafica;
- simulazioni Monte Carlo;
- comparativa statica numerica;
- esperimenti agent-based.

Terzo, e' fortemente interdisciplinare, perche' collega economia industriale, diffusione tecnologica, contagio sociale, teoria delle piattaforme e sistemi complessi.

Quarto, rende molto visibile il legame tra eterogeneità individuale e risultati aggregati.

18. Conclusione

Le esternalità di rete trasformano il problema della domanda in un problema di coordinamento. L'utilità individuale dipende dalla diffusione aggregata, e la diffusione aggregata dipende a sua volta dalle decisioni individuali. Da questa interdipendenza emergono soglie, non linearità, molteplicità di equilibri e dipendenza dalle condizioni iniziali.

Per questo motivo il modello di adozione con esternalità di rete e' un eccellente caso di studio per un corso di metodi computazionali per modelli stocastici e dinamiche collettive. E' abbastanza semplice da essere implementato subito, ma abbastanza ricco da sostenere discussioni serie su coordinamento, tipping, lock-in, diffusione e strategie di intervento.

19. Bibliografia minima

1. Katz, M. L., and Shapiro, C. (1985). Network Externalities, Competition, and Compatibility. *American Economic Review*, 75(3), 424-440.
2. Farrell, J., and Saloner, G. (1985). Standardization, Compatibility, and Innovation. *RAND Journal of Economics*, 16(1), 70-83.
3. Economides, N. (1996). The Economics of Networks. *International Journal of Industrial Organization*, 14(6), 673-699.
4. Shy, O. (2001). *The Economics of Network Industries*. Cambridge University Press.
5. Rogers, E. M. (2003). *Diffusion of Innovations*. Free Press.

Appendice A. Implementazione in Python: struttura del codice

Questa appendice propone una struttura semplice per implementare il modello di adozione con esternalità di rete in Python. L'obiettivo non è costruire un programma sofisticato, ma fornire una guida leggibile anche da chi usa altri linguaggi di programmazione.

La logica è la stessa usata nella dispensa:

1. definire la funzione di rete $f(q)$;
2. costruire la mappa aggregata $T(q)$;
3. calcolare i punti fissi con una iterazione numerica;
4. confrontare il risultato con una simulazione agent-based;
5. visualizzare i risultati con grafici semplici.

Il codice è volutamente elementare:

- cicli espliciti;

- nomi leggibili;
- poche librerie;
- funzioni corte e facili da modificare.

A.1 Librerie minime

Per una prima implementazione bastano queste librerie:

```
import random
import matplotlib.pyplot as plt
```

Se si vuole calcolare una media in modo comodo, si può aggiungere:

```
import statistics
```

Quindi:

- `random` serve per estrarre i parametri individuali;
- `matplotlib.pyplot` serve per i grafici;
- `statistics` è opzionale.

Non è necessario usare `numpy` in una prima versione.

A.2 Idea generale dell'implementazione

Ci sono due versioni del modello.

Versione aggregata

Si lavora direttamente sulla quota di adottanti q e si itera la mappa

$$q_{t+1} = T(q_t).$$

Versione agent-based

Si generano N agenti con parametri individuali β_i e si applica la regola di adozione a ciascun agente:

$$\beta_i f(q_t) \geq p.$$

Le due versioni sono strettamente collegate. La versione aggregata è più compatta. La versione agent-based è più vicina all'intuizione microscopica.

A.3 La funzione di rete

Conviene partire da una funzione Python che rappresenti $f(q)$.

Caso lineare

```
def network_effect(q):  
    return q
```

Questo è il caso più semplice:

$$f(q) = q.$$

Versione un po' più generale

Se si vuole cambiare forma senza riscrivere il resto del codice, si può usare:

```
def network_effect(q, mode="linear", alpha=1.0):  
    if mode == "linear":  
        return q  
    elif mode == "power":  
        return q ** alpha  
    elif mode == "saturation":  
        return q / (alpha + q)  
    else:  
        return q
```

Interpretazione:

- `linear` corrisponde a $f(q) = q$;
- `power` corrisponde a $f(q) = q^\alpha$;
- `saturation` introduce una crescita che rallenta.

Per il primo laboratorio conviene usare solo il caso lineare.

A.4 La mappa aggregata $T(q)$

Nel caso in cui β sia uniforme su $[0, B]$, la quota attesa di adottanti è

$$T(q) = 1 - \frac{p}{Bf(q)},$$

quando il valore è ben definito. Per ragioni numeriche conviene sempre forzare il risultato nell'intervallo $[0, 1]$.

```
def adoption_map(q, p, B, mode="linear", alpha=1.0):  
    f_q = network_effect(q, mode=mode, alpha=alpha)  
  
    if f_q <= 0:  
        return 0.0  
  
    value = 1.0 - p / (B * f_q)
```

```

if value < 0.0:
    return 0.0
elif value > 1.0:
    return 1.0
else:
    return value

```

Questa funzione implementa direttamente la mappa di punto fisso.

A.5 Iterazione di punto fisso

Una volta definita la mappa, si può costruire una funzione che iteri il sistema fino a convergenza.

```

def fixed_point_iteration(q0, p, B, mode="linear", alpha=1.0,
                          tolerance=1e-8, max_steps=1000):
    q = q0
    history = [q]

    for step in range(max_steps):
        q_new = adoption_map(q, p, B, mode=mode, alpha=alpha)
        history.append(q_new)

        if abs(q_new - q) < tolerance:
            return q_new, history

    q = q_new

    return q, history

```

La funzione restituisce:

- il valore finale della quota di adozione;
- la traiettoria completa delle iterazioni.

Questo è utile perché permette sia di trovare il punto fisso sia di studiare la dinamica di convergenza.

A.6 Primo esempio completo

```

p = 0.2
B = 1.0
q0 = 0.1

q_star, history = fixed_point_iteration(q0, p, B)

```

```
print("Equilibrio trovato:", q_star)
print("Numero di iterazioni:", len(history) - 1)
```

Questo è il primo test minimo da fare.

A.7 Grafico della traiettoria temporale

Per visualizzare come converge l'iterazione, si può usare:

```
def plot_history(history, title="Traiettoria di adozione"):
    times = list(range(len(history)))

    plt.plot(times, history)
    plt.xlabel("tempo")
    plt.ylabel("q")
    plt.title(title)
    plt.ylim(0.0, 1.0)
    plt.show()
```

Esempio:

```
plot_history(history, title="Iterazione di punto fisso")
```

A.8 Grafico di $T(q)$ e della retta identità

Questo è uno dei grafici più importanti del caso di studio, perché mostra direttamente i punti fissi come intersezioni tra la curva $T(q)$ e la retta q .

```
def plot_map(p, B, mode="linear", alpha=1.0, num_points=200):
    q_values = []
    T_values = []

    for n in range(num_points + 1):
        q = n / num_points
        T_q = adoption_map(q, p, B, mode=mode, alpha=alpha)
        q_values.append(q)
        T_values.append(T_q)

    plt.plot(q_values, T_values, label="T(q)")
    plt.plot(q_values, q_values, label="q")
    plt.xlabel("q")
    plt.ylabel("valore")
    plt.title("Mappa di adozione e retta identità")
    plt.ylim(0.0, 1.0)
    plt.legend()
    plt.show()
```


Esempio:

```
plot_map(p=0.2, B=1.0)
```

A.9 Cercare equilibri multipli

Un modo semplice per esplorare la presenza di più equilibri è ripetere l'iterazione partendo da valori iniziali diversi.

```
def scan_initial_conditions(p, B, mode="linear", alpha=1.0,
                           num_initial_points=21,
                           tolerance=1e-6, max_steps=1000):

    equilibria = []

    for n in range(num_initial_points):
        q0 = n / (num_initial_points - 1)
        q_star, history = fixed_point_iteration(
            q0=q0,
            p=p,
            B=B,
            mode=mode,
            alpha=alpha,
            tolerance=tolerance,
            max_steps=max_steps
        )
        equilibria.append(q_star)

    return equilibria
```

Se si vogliono eliminare duplicati numerici quasi uguali:

```
def unique_values(values, tolerance=1e-5):
    unique = []

    for x in values:
        already_present = False
        for y in unique:
            if abs(x - y) < tolerance:
                already_present = True
                break
        if not already_present:
            unique.append(x)

    return unique
```

Esempio:

```
equilibria = scan_initial_conditions(p=0.2, B=1.0)
unique_equilibria = unique_values(equilibria)
```

```
print("Equilibri trovati:", unique_equilibria)
```

Questo metodo non è sofisticato, ma è molto utile in un laboratorio didattico.

A.10 Curva prezzo-adozione

Per studiare la comparativa statica rispetto al prezzo, si può risolvere il punto fisso per molti valori di p .

```
def price_penetration_curve(price_values, B, q0=0.1,
                             mode="linear", alpha=1.0):
    q_stars = []

    for p in price_values:
        q_star, history = fixed_point_iteration(
            q0=q0,
            p=p,
            B=B,
            mode=mode,
            alpha=alpha
        )
        q_stars.append(q_star)

    return q_stars
```

E poi fare il grafico:

```
def plot_price_penetration(price_values, q_values):
    plt.plot(price_values, q_values)
    plt.xlabel("prezzo p")
    plt.ylabel("adozione di equilibrio q")
    plt.title("Curva prezzo-adozione")
    plt.ylim(0.0, 1.0)
    plt.show()
```

Esempio completo:

```
price_values = [0.02 * n for n in range(1, 41)]
q_values = price_penetration_curve(price_values, B=1.0, q0=0.1)

plot_price_penetration(price_values, q_values)
```

A.11 Versione agent-based

Passiamo ora alla simulazione microscopica.

Ogni agente i riceve un parametro individuale β_i estratto da una distribuzione uniforme su $[0, B]$.

Generazione della popolazione

```
def create_population(N, B):
    betas = []

    for i in range(N):
        beta_i = random.uniform(0.0, B)
        betas.append(beta_i)

    return betas
```

Stato iniziale di adozione

Per semplicità, rappresentiamo gli agenti con una lista di zeri e uni:

- 0 = non adotta
- 1 = adotta

```
def create_initial_adoption(N, initial_fraction=0.0):
    adoption = []

    for i in range(N):
        u = random.random()
        if u < initial_fraction:
            adoption.append(1)
        else:
            adoption.append(0)

    return adoption
```

Quota di adottanti

```
def adoption_fraction(adoption):
    return sum(adoption) / len(adoption)
```

A.12 Aggiornamento degli agenti

In una versione molto semplice, a ogni passo ogni agente ricalcola la propria decisione dato il valore corrente di q .

```

def update_agents(betas, adoption, p, mode="linear", alpha=1.0):
    q = adoption_fraction(adoption)
    f_q = network_effect(q, mode=mode, alpha=alpha)

    new_adoption = []

    for beta_i in betas:
        if beta_i * f_q >= p:
            new_adoption.append(1)
        else:
            new_adoption.append(0)

    return new_adoption

```

Questa regola implementa direttamente la soglia di adozione.

A.13 Una simulazione agent-based completa

```

def run_agent_based_simulation(N, B, p, T,
                              initial_fraction=0.0,
                              mode="linear", alpha=1.0):
    betas = create_population(N, B)
    adoption = create_initial_adoption(N, initial_fraction=initial_fraction)

    history_q = [adoption_fraction(adoption)]

    for t in range(T):
        adoption = update_agents(
            betas=betas,
            adoption=adoption,
            p=p,
            mode=mode,
            alpha=alpha
        )
        history_q.append(adoption_fraction(adoption))

    results = {
        "betas": betas,
        "final_adoption": adoption,
        "history_q": history_q
    }

    return results

```

Esempio:

```

results = run_agent_based_simulation(
    N=1000,
    B=1.0,
    p=0.2,
    T=50,
    initial_fraction=0.2
)

print("Adozione finale:", results["history_q"][-1])
plot_history(results["history_q"], title="Simulazione agent-based")

```

A.14 Ripetere molte simulazioni

Per studiare gli effetti di popolazione finita conviene ripetere l'esperimento più volte.

```

def run_many_agent_simulations(num_runs, N, B, p, T,
                               initial_fraction=0.0,
                               mode="linear", alpha=1.0):

    final_q_values = []
    all_histories = []

    for run in range(num_runs):
        results = run_agent_based_simulation(
            N=N,
            B=B,
            p=p,
            T=T,
            initial_fraction=initial_fraction,
            mode=mode,
            alpha=alpha
        )

        final_q_values.append(results["history_q"][-1])
        all_histories.append(results["history_q"])

    summary = {
        "final_q_values": final_q_values,
        "all_histories": all_histories
    }

    return summary

```

Se si vuole la media finale:

```
def mean_value(values):
    return sum(values) / len(values)
```

Esempio:

```
summary = run_many_agent_simulations(
    num_runs=20,
    N=500,
    B=1.0,
    p=0.2,
    T=50,
    initial_fraction=0.2
)

print("Media finale:", mean_value(summary["final_q_values"]))
```

A.15 Confronto tra versione aggregata e versione agent-based

Il confronto più naturale consiste nel verificare se la simulazione microscopica converge vicino al punto fisso della versione aggregata.

```
p = 0.2
B = 1.0
q0 = 0.2

q_star, history_agg = fixed_point_iteration(q0=q0, p=p, B=B)

results_abm = run_agent_based_simulation(
    N=1000,
    B=B,
    p=p,
    T=50,
    initial_fraction=q0
)

print("Equilibrio aggregato:", q_star)
print("Equilibrio agent-based:", results_abm["history_q"][-1])
```

Questo è un passaggio didatticamente importante, perché mostra il collegamento tra descrizione macroscopica e simulazione di agenti.

A.16 Esperimento sulla massa critica iniziale

Per studiare l'effetto del seme iniziale si può ripetere la simulazione per diversi valori della frazione iniziale.

```

def seed_experiment(seed_values, N, B, p, T,
                    mode="linear", alpha=1.0):
    final_values = []

    for seed in seed_values:
        results = run_agent_based_simulation(
            N=N,
            B=B,
            p=p,
            T=T,
            initial_fraction=seed,
            mode=mode,
            alpha=alpha
        )
        final_values.append(results["history_q"][-1])

    return final_values

```

Grafico:

```

def plot_seed_experiment(seed_values, final_values):
    plt.plot(seed_values, final_values)
    plt.xlabel("frazione iniziale di adottanti")
    plt.ylabel("adozione finale")
    plt.title("Effetto della massa critica iniziale")
    plt.ylim(0.0, 1.0)
    plt.show()

```

Esempio:

```

seed_values = [0.05 * n for n in range(11)]
final_values = seed_experiment(
    seed_values=seed_values,
    N=1000,
    B=1.0,
    p=0.2,
    T=50
)

plot_seed_experiment(seed_values, final_values)

```

A.17 Organizzazione consigliata del file

Per tenere il codice leggibile, conviene organizzare il file in questo ordine:

1. import delle librerie;
2. funzione `network_effect`;

3. funzioni per la versione aggregata:

- `adoption_map`
- `fixed_point_iteration`
- funzioni di grafico

4. funzioni per la versione agent-based:

- `create_population`
- `create_initial_adoption`
- `update_agents`
- `run_agent_based_simulation`

5. blocco finale con gli esperimenti.

Per esempio:

```
if __name__ == "__main__":
    p = 0.2
    B = 1.0

    q_star, history = fixed_point_iteration(q0=0.1, p=p, B=B)
    print("Equilibrio aggregato:", q_star)
    plot_history(history, title="Iterazione di punto fisso")

    results = run_agent_based_simulation(
        N=1000,
        B=B,
        p=p,
        T=50,
        initial_fraction=0.2
    )
    print("Equilibrio agent-based:", results["history_q"][-1])
    plot_history(results["history_q"], title="Simulazione agent-based")
```

A.18 Estensioni semplici del codice

Una volta che la struttura base funziona, ci sono alcune estensioni naturali.

Funzione di rete diversa

Basta modificare `network_effect`.

Distribuzione di β non uniforme

Si può sostituire `random.uniform(0.0, B)` con un'altra distribuzione.

Adozione reversibile o inerziale

Si può fare in modo che gli agenti non cambino stato istantaneamente, ma con una certa probabilità di aggiornamento.

Due piattaforme in competizione

Si possono introdurre due quote di mercato, due prezzi e due effetti di rete.

Effetti di rete locali

Si può sostituire la quota aggregata q con la quota di vicini adottanti in una rete esplicita.

A.19 Commento finale

La struttura proposta qui ha due vantaggi.

Primo, è abbastanza semplice da essere letta quasi come pseudocodice da chi programma in C, Julia, R, Matlab o altri linguaggi.

Secondo, è abbastanza vicina a Python reale da poter essere eseguita con pochissime modifiche da chi conosce già Python.

Il punto metodologico importante è questo: anche un modello molto semplice di esternalità di rete consente già di lavorare su

- equilibri come punti fissi;
- dinamiche di convergenza;
- eterogeneità individuale;
- simulazioni Monte Carlo;
- comparativa statica numerica.

Per questo motivo è un ottimo caso di studio per un corso di metodi computazionali.